



DO NOT TOUCH THE RULER, CHANGE FONT, SIZE!

JB XX – SQL Code and Database Objects Standards

1. Indentation

- 1.1. Indentation is done using tab
- 1.2. Indent main (SELECT, WHERE, FROM, etc.) keywords by one tab
- 1.3. AND, OR should have column names as “=” after the name aligned
- 1.4. Columns in select part are separated by “,” at the beginning of the line and intended bellow
SELECT
- 1.5. Keywords specifying SELECT behavior are on the same line as SELECT/UPDATE/DELETE.
Keywords like: TOP, DISTINCTS, etc.
- 1.6. Each join predicate is intended to be 1 tab to right from joined object.
- 1.7. Each join predicate is aligned to the rest of the join predicates within the same join.
- 1.8. Each join predicate is on a separate line.
- 1.9. Each query predicate is on separate line, aligned with the other predicated and inteded by 1 tab.

```
SELECT TOP(10)
    vessel_name = ves.[Vessel_Name]
    , vessel_id  = ves.[Vessel_ID]
FROM [dbo].[LIB_VESSELS] ves
WHERE ves.[Vessel_ID]      = 10
      AND ves.[Vessel_Code] = 'aa'
      AND (
            ves.[Vessel_Name] = 'abc'
          OR ves.[FleetCode]   = 5
        )
```

- 1.10. Example:

2. Formatting

- 2.1. SQL keywords are UPPERCASE.
 - For example: SELECT, WHERE, OR, AND
- 2.2. All objects need to follow 2 name naming convention. 1 name naming convention is prone to defaults behavior issues.

- For example, [dbo].[LIB_Vessels].

2.3. Objects and columns are quoted using [] brackets

- For Example: [dbo].[lib_vessels]

2.4. Each column is on separate line

3. Naming conventions

3.1. Characters allowed in the names:

- Letters
- Numbers
- Underscore _

3.2. Names can start only with letters

3.3. Each object in query has to have an alias. The only exception is single object query.

3.4. Each column in a query has to be using aliased 2 name naming convention.

- If lib_vessels is aliased with “lvs”, the column name would be written as lvs.name

3.5. Objects (like tables, vies, functions, procedures, ...), columns names follow snake_case

- For example: lib_vessels, fn_get_vessel_name(), sp_update_index

3.6. Object except tables must have a prefix describing their type:

- Stored procedure **sp**_procedure
- View **vw**_view
- Scalar valued function **fn**_function
- Table value function **tvf**_function
- Index **ix**_index or **cx**_index
- Primary key **pk**_key
- Foreign key **fk**_key
- Unique constraint **uq**_constraint
- Default constraint **df**_constraint

3.7. Index, primary key, unique constrain furthermore require additional naming

- Index type.
 - 3.7..1. rowstore clustered index starts with cx_
 - 3.7..2. rowstore nonclustered index starts with ix_
 - 3.7..3. Columnstore clusterd index startwth with ccx_
 - 3.7..4. Columnstore nonclustered index start with cnx_
- For primary key and unique constraints c or n needs to be added to define underlying index type.
- Object schema name
- Object name
- Indexed columns in order as they are defined
- Examples:
 - 3.7..1. Index: ix_dbo_lib_vessels_vessel_name
 - 3.7..2. Primary key: pkc_dbo_lib_vessels_uid
 - 3.7..3. Unique constraint uqn_dbo_lib_vessels_uid

3.8. Foreign key furthermore requires additional information

- Table utilizing foreign key
- Table which is being referenced by the key
- Columns which are referenced
- Additional specifically defined features of the index. For example ON DELETE
- Example: fk_dbo_voyage_dbo_lib_vessels_uid_delete

3.9. Default constraint furthermore requires additional information

- Table constraint is applied on
- Column constraint is applied on
- Example: df_dbo_lib_vessels_install_status

3.10. All constraints have to have a name defined. Automatically assigned names are not allowed.

4. Coding standards – refrain from:

4.1. Using merge

- It has plenty of bugs ranging from self deadlocks through disregarding PK to not updating data for underlying data of indexed view.
- Replace with separate insert/update/delete

4.2. Using cursor unless necessary.

- Business logic can often be written in correct set based logic.
- Cursors introduce logical fragmentation even for seemingly unaffected data
- If tables have triggers, every iteration of the cursor triggers the trigger on a table. Set based logic triggers it only once.
- If necessary consider using WHILE loop instead

4.3. Populating data in truncate – insert method on big datasets. Use incremental logic

4.4. Heavy triggers – they should be light and have minimal impact. No stored procedures executing data per row.

4.5. Per row logic in general – Select runs internal cursor to get the data, so running selects for each row separately defies the logic running the

4.6. Type conversions in joins and predicates – it breaks search argument ability condition and hence cause usage of wrong index or index scan

4.7. Scalar function and build in functions like LEFT, RIGHT – for the same reason

4.8. Using * in the select statements.

5. Coding standard – make sure:

5.1. For multi-statement transactions set XACT_ABORT = ON

5.2. Multi-statement transactions should follow statement order of extracting data first and then performing modifications.

5.3. Use CTEs over nested queries

5.4. Always extract the minimal dataset required.

- 5.5. The code is well documented, and complicated statements are explained. Explain the purpose of the statement. If 5 tables are being joined the comment should explain what We want to achieve. Not informing that 5 tables are joined.
- 5.6. Prefer temporary tables over table variables.
- 5.7. Every query has fully covering index. Exception can apply for indexes covering significant portion of table, when key lookup might be needed.